

Practica 1, primer proyecto java

1. Abran NetBeans.
2. Hagan clic en **File → New Project**.
3. Seleccionen **Java Application**.
4. Coloquen como nombre del proyecto: ClasePOO
5. Finalicen la creación del proyecto.

Parte 2. Crear nuestra primera clase

1. En el panel izquierdo, localicen el nodo **Source Packages**.
2. Expandan el paquete del proyecto.
3. Hagan clic derecho sobre el paquete.
4. Seleccionen:
New → Java Class
5. Escriban como nombre: Vehiculo

Después de crearla tendrán algo parecido a esto:

```
public class Vehiculo {  
  
  
  
  
  
  
}
```

Parte 3. Agregar atributos

```
public class Vehiculo {
```

```
    String marca;
```

```
    String modelo;
```

```
    int anio;
```

```
    double precio;
```

```
}
```

Parte 4. Crear un método

Pegar este método en la clase Vehiculo

```
void mostrarInformacion() { System.out.println("Marca: " + marca);  
System.out.println("Modelo: " + modelo); System.out.println("Año: " + anio);  
System.out.println("Precio: Q" + precio); }
```

así esta nuestra clase:

OBJETO

|—— Estado → atributos

|—— Comportamiento → métodos

Parte 5. Crear nuestro primer objeto

Ahora regresen a la clase principal.

Puedes indicar:

1. En el panel izquierdo, busquen la clase que contiene el método main.
2. Hagan doble clic sobre ella.
3. Dentro del método main, escriban:

```
public static void main(String[] args) {
```

```
Vehiculo carro1 = new Vehiculo();
```

```
}
```

Parte	Significado
Vehiculo	Tipo de dato
carro1	Nombre de la variable
new	Crea un nuevo objeto
Vehiculo()	Llama al constructor

```
Vehiculo carro1 = new Vehiculo();
```

Parte 6. Asignar valores al objeto

```
public static void main(String[] args) {
```

```
    Vehiculo carro1 = new Vehiculo();
```

```
    carro1.marca = "Toyota";
```

```
    carro1.modelo = "Corolla";
```

```
    carro1.anio = 2025;
```

```
    carro1.precio = 185000;
```

```
}
```

¿Dónde están almacenados estos valores?

Imprimamos los valores:

```
carro1.mostrarInformacion();
```

debería quedar el método main así:

```
public static void main(String[] args) {
```

```
    Vehiculo carro1 = new Vehiculo();
```

```
    carro1.marca = "Toyota";
```

```
    carro1.modelo = "Corolla";
```

```
    carro1.anio = 2025;
```

```
    carro1.precio = 185000;
```

```
    carro1.mostrarInformacion();
```

```
}
```

Parte 7:Y si creamos mas vehículos:

```
public static void main(String[] args) {
```

```
    Vehiculo carro1 = new Vehiculo();
```

```
    carro1.marca = "Toyota";
```

```
    carro1.modelo = "Corolla";
```

```
    carro1.anio = 2025;
```

```
    carro1.precio = 185000;
```

```
    Vehiculo carro2 = new Vehiculo();
```

```
    carro2.marca = "Jeep";
```

```
    carro2.modelo = "Wrangler TJ";
```

```
    carro2.anio = 2001;
```

```
    carro2.precio = 75000;
```

```
    carro1.mostrarInformacion();
```

```
    System.out.println("-----");
```

```
carro2.mostrarInformacion();
}
```

Paso 8:

Que es el constructor? , primero regresemos al paso 5,

```
Vehiculo carro1 = new Vehiculo();
```

Si nos damos cuenta tiene (); como si fuera un método, entonces “El constructor es el mecanismo que se ejecuta automáticamente cuando creamos un objeto con new. Su trabajo principal es darle valores iniciales al objeto.”

El constructor se debe llamar igual que la clase . como es un método puede o no recibir parámetros, este es el constructor para la clase Vehiculos (pegar en la clase vehículos)

```
public Vehiculo(String marca, String modelo, int anio) {  
    this.marca = marca;  
    this.modelo = modelo;  
    this.anio = anio;  
}
```

Parte 9. Explicar this

this significa “este objeto”.

Cuando estamos dentro de una clase, `this` sirve para referirnos al objeto actual, es decir, al objeto que está ejecutando ese código.

//en la clase vehículo, tenemos el codigo así,:

String marca;

```
public Vehiculo(String marca, String modelo, int anio) {
```

```
this.marca = marca;
```

//existen 2 variables que se llama marca, pero el qui tiene la palabra this es para el atributo o variable global

this.marca = marca;

```
|  
|  
|_ parámetro recibido
```

|
└─ atributo del objeto actual

Parte 10. Introducir encapsulamiento

Hasta donde llevamos nuestro proyecto cualquiera puede hacer cambio a nuestros atributos, ejemplo podemos poner :

```
carro1.precio = -500000;
```

¿Tiene sentido que un vehículo tenga precio negativo?

Podemos poner una validación cuando se envia y eso quiere decir que cada vez que se envia un nuevo valor de cualquier lugar se tiene que poner la misma validación, o podemos poner una sola validación de cuando se recibe.

Para que nadie los pueda modificar desde afuera agregamos el “private” en nuestros atributos así:

```
private String marca;  
private String modelo;  
private int anio;  
private double precio;
```

debería dar error!!

Parte 11. Encapsulamiento :significa proteger el estado interno de un objeto y controlar cómo se accede o modifica.

Para controlar el acceso a lectura y editar se usan los métodos getters y setters, get es para obtener y set es para dar un nuevo valor.

En este caso debemos de controlar los valores que nos dan, entonces usamos un set al precio así:

```
//pegar en la clase vehiculo  
public void setPrecio(double precio) {  
  
    if (precio > 0) {  
        this.precio = precio;  
    } else {  
        System.out.println("Error: el precio debe ser mayor que cero");  
    }  
}
```

```

    }
}

```

```

public String getMarca(){
    return this.marca;
}

```

Y en la clase principal

```

Reemplazar carro1.precio=-1000; por:
carro1.setPrecio(-1000);

```

Ejercicio POO: agregar llantas a un vehículo

Objetivo

Crear una clase llamada Llanta y utilizarla como atributo dentro de la clase Vehiculo.

- 1) Crear la clase Llanta
- 2) Agregar los atributos de la llanta

Dentro de la clase Llanta, agregue los siguientes atributos privados:

```

private String marca;
private int tamano;
private double presion;

```

- 3) Crear el constructor de Llanta recibiendo los 3 valores como parámetro
- 4) Crear un método para mostrar la información

```

public void mostrarInformacion() { System.out.println("Marca de
llanta: " + marca); System.out.println("Tamaño: " + tamano + "
pulgadas"); System.out.println("Presión: " + presion + " PSI"); }

```

- 5) Agregar una llanta a la clase Vehiculo

Dentro de la clase Vehiculo, agregue un nuevo atributo:

```

private Llanta llanta;

```

significa que la clase Vehiculo tendrá una variable que puede guardar un objeto de tipo Llanta.

6) Modificar el constructor de Vehiculo

Busque el constructor de Vehiculo.

```
public Vehiculo(  
    String marca,  
    String modelo,  
    int anio,  
    double precio,  
    Llanta llanta) {  
  
    this.marca = marca;  
    this.modelo = modelo;  
    this.anio = anio;  
    this.precio = precio;  
    this.llanta = llanta;  
}
```

//al cambiar el constructor se espera q de error

7) Mostrar la información de la llanta desde Vehiculo

Busque el método: `mostrarInformacion()` en la clase Vehiculo

Agregue al final:

```
System.out.println("Información de la llanta:");
```

```
llanta.mostrarInformacion();
```

8) Crear un vehículo con una llanta,

Ahora antes de crear un nuevo vehículo debemos de crear la llanta

Antes de crear el vehículo jeep se pega la línea:

```
Llanta llantaJeep = new Llanta( "BFGoodrich", 31, 32 );
```

//debe dar error

el error es por que falta el constructor de llanta que reciba parametros, crear el constructor en el archivo de llanta que reciba los 3 parametros.

```
Vehiculo carro2 = new Vehiculo( "Jeep", "Wrangler TJ", 2001,  
75000,llantaJeep );
```


Hacer lo mismo con el carro1

Crear un JFrame Form

para empezar esta parte se debio haber finalizado las practicas anteriores:

3. En el panel izquierdo, expanda:

Source Packages

4. Localice el paquete donde se encuentran las clases:

Vehiculo

Llanta

3. En el panel izquierdo, expanda:

Source Packages

4. Localice el paquete donde se encuentran las clases:

Vehiculo

Llanta

5. Haga clic derecho sobre el paquete.
6. Seleccione:

New → Java Class

y escribir el nombre de la nueva clase: FrmVehiculo

```
public class FrmVehiculo {  
  
}
```

7. ahora necesitamos que la clase sea un formulario:

```
import javax.swing.JFrame;
```

```
public class FrmVehiculo extends JFrame {  
  
}
```

significa que **FrmVehiculo** hereda las características de **JFrame**.

8. Crear el constructor

Agregar el constructor, recordemos que el constructor se debe llamar igual que la clase:

```
public FrmVehiculo() {  
  
}
```

Ahora configurar la ventana:

@bloque de código

```
public FrmVehiculo() {  
    setTitle("Registro de Vehículo");  
    setSize(500, 400);  
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    setLocationRelativeTo(null);  
    setLayout(null);  
}
```

ahora que significa:

Título: definimos el título así

```
    setTitle("Registro de Vehículo");
```

Tamaño: es el tamaño del formulario dado en pixeles

```
setSize(500, 400);
```

Significa aproximadamente:

500 píxeles de ancho

400 píxeles de alto

Cerrar el programa

```
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

Centrar ventana

```
setLocationRelativeTo(null);
```

Ubicación manual

Esta es la importante para tu objetivo:

```
setLayout(null);
```

Estamos diciéndole a Java que nosotros vamos a decidir exactamente dónde estará cada componente.

9. ahora llamemos a nuestro formulario

nos vamos a la clase principal la que se llama igual que el proyecto y tiene un método main, dentro del método main se coloca:

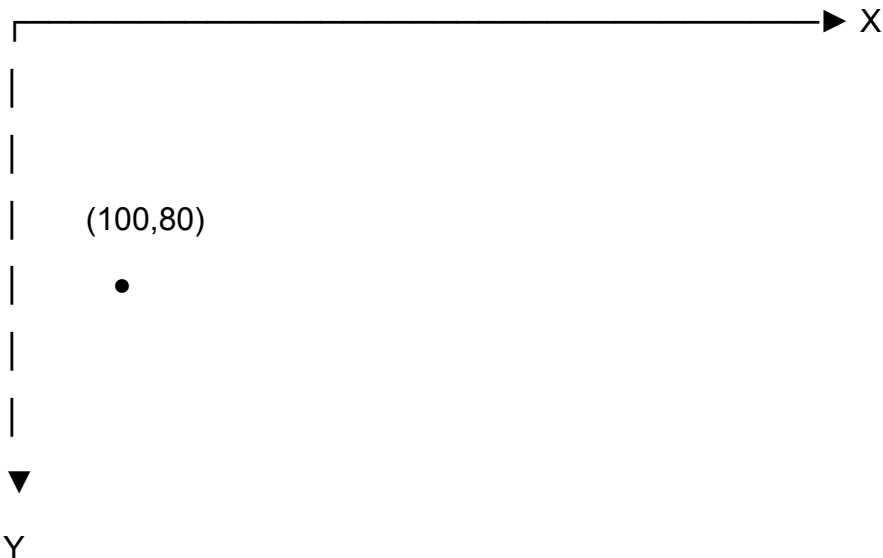
```
FrmVehiculo ventana = new FrmVehiculo();
```

```
ventana.setVisible(true);
```

10. el sistema de coordenadas

las coordenadas en el formulario funcionan así:

(0,0)



La esquina superior izquierda es:

$X = 0$

$Y = 0$

X aumenta hacia la derecha.

Y aumenta hacia abajo.

Este concepto será fundamental para entender:

```
setBounds(x, y, ancho, alto);
```

11. Crear el primer JLabel

Ahora agregar:

```
import javax.swing.JLabel;
```

afuera como que si fuera una variable global:

```
JLabel lblTitulo = new JLabel("REGISTRO DE VEHÍCULO");
```

Dentro del constructor:

```
lblTitulo.setBounds(150, 20, 200, 30);
```

```
add(lblTitulo);
```

¿Qué significan estos cuatro números?

```
setBounds(150, 20, 200, 30);
```

150 → posición X

20 → posición Y

200 → ancho

30 → alto

12. Crear el campo Marca

afuera del constructor como global

```
JLabel lblMarca = new JLabel("Marca:");
```

adentro del constructor:

```
lblMarca.setBounds(50, 70, 100, 30);
```

```
add(lblMarca);
```

13. Ahora el campo de texto

agregamos el import

```
import javax.swing.JTextField;
```

afuera del constructor

```
JTextField txtMarca = new JTextField();
```

adentro del constructor

```
txtMarca.setBounds(150, 70, 200, 30);
```

```
add(txtMarca);
```

14. agregamos modelo

15. afuera del constructor

```
JLabel lblModelo = new JLabel("Modelo:");
```

```
JTextField txtModelo = new JTextField();
```

adentro del constructor

```
lblModelo.setBounds(50, 110, 100, 30);
```

```
add(lblModelo);
```

```
txtModelo.setBounds(150, 110, 200, 30);
```

```
add(txtModelo);
```

16. Agregar Año

afuera del constructor

```
JLabel lblAnio = new JLabel("Año:");
```

```
JTextField txtAnio = new JTextField();
```

adentro del constructor

REGISTRO DE VEHÍCULO	
Marca:	
Modelo:	
Año:	
Precio:	
[GUARDAR]	

19. Unir formulario con OBJETOS

para este punto deben haber completado el crear las clases:vehiculo y llantas y el formulario

hacer funcionar el botón

agregamos el siguiente codigo

```
btnGuardar.addActionListener(e -> {
```

```
    System.out.println("Presionó el botón Guardar");
```



```
});
```

¿Qué es un evento?

Un evento es una acción que ocurre mientras el programa se está ejecutando.

Algunos ejemplos de eventos:

El usuario hace clic en un botón

El usuario escribe en un campo

El usuario mueve el mouse

El usuario cierra una ventana

El usuario selecciona una opción

que es `addActionListener` ?

al objeto `JButton` se debe de agregar el evento, que significa `addActionListener` se encarga de escuchar acciones como el clic de un botón.

¿Qué significa la flecha?

```
e -> {
```

Esta sintaxis se conoce como **expresión lambda**.

La podemos leer así:

Cuando ocurra el evento, ejecute el código que está dentro de las llaves.

```
btnGuardar.addActionListener(e -> {
```

```
// Este código se ejecuta cuando hacen clic
```

```
});
```

20. Obtener lo escrito

para obtener el texto de un teetextbox se usa el metodo "getText();" asi:

```
btnGuardar.addActionListener(e -> {  
    //el codigo se ejecuta cuando hacen clic  
    String marca = txtMarca.getText();  
    String modelo = txtModelo.getText();  
  
    System.out.println("Marca: " + marca);  
    System.out.println("Modelo: " + modelo);  
});
```

solo estamos obteniendo el texto falta los números ya que estamos diciendo

getText (obtener texto)

21. Obtener números usando parseo

se utilizan métodos que llevan la palabra "parse" los cuales ayuda a convertir de un tipo a otro

para el año seria asi:

```
int anio = Integer.parseInt(txtAnio.getText());
```

Para el precio:

```
double precio =
```

```
    Double.parseDouble(txtPrecio.getText());
```

quedando el metodo asi:

```

btnGuardar.addActionListener(e -> {

    String marca = txtMarca.getText();

    String modelo = txtModelo.getText();

    int anio =

        Integer.parseInt(txtAnio.getText());

    double precio =

        Double.parseDouble(txtPrecio.getText());

    Vehiculo vehiculo =

        new Vehiculo(

            marca,

            modelo,

            anio,

            precio

        );

    vehiculo.mostrarInformacion();

});

```

22. Mostrar un mensaje en pantalla

agreguemos lo siguiente al final del método anterior

```

JOptionPane.showMessageDialog( this, "Vehículo guardado correctamente" );

```

Diagrama de clases

¿Qué es UML?

UML significa **Lenguaje Unificado de Modelado**. Es una forma visual de representar cómo estará organizado y cómo funcionará un sistema antes o durante su programación.

UML utiliza diagramas con símbolos, cajas, líneas y flechas para facilitar la comprensión del software.

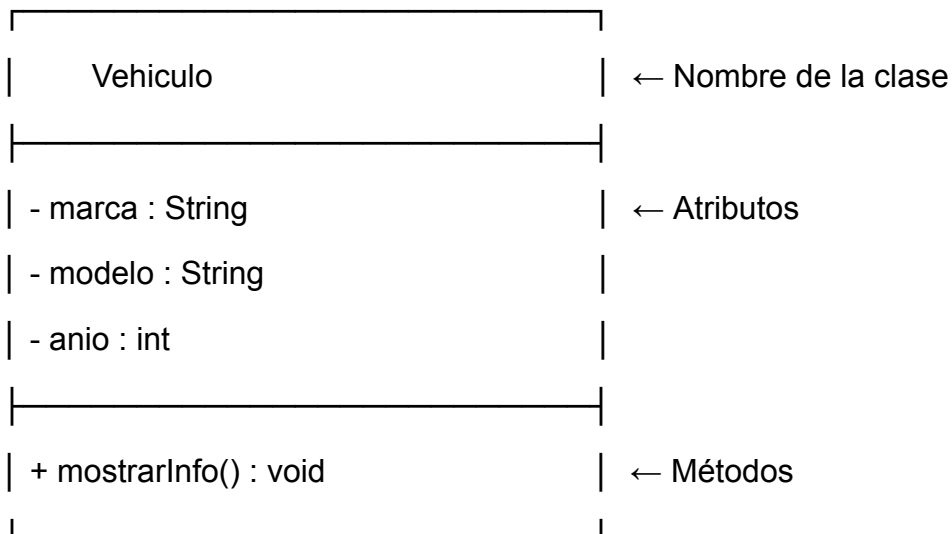
UML = lenguaje visual para diseñar y explicar sistemas

¿Qué es un diagrama de clases?

Un **diagrama de clases** es un tipo de diagrama UML que muestra:

- Las clases del sistema.
- Sus atributos.
- Sus métodos.
- Las relaciones entre las clases.

Una clase normalmente se representa mediante una caja dividida en tres partes:



Los símbolos indican la visibilidad:

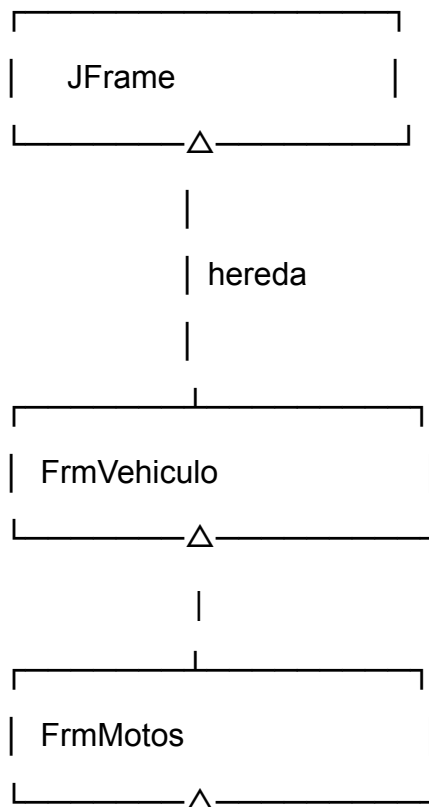
- privado
- + público
- # protegido

1. **Herencia:** `FrmVehiculo` hereda de `JFrame`

```
public class FrmVehiculo extends JFrame {  
    }  
}
```

`FrmVehiculo` es una ventana.

Gracias a la herencia, `FrmVehiculo` puede utilizar métodos que ya existen en `JFrame`, por ejemplo:



2. **Composición:** `Vehiculo` tiene `llantas`

En la clase `Vehiculo` podríamos tener:

```
private Llanta llanta;
```

O, para representar las cuatro llantas:

```
private Llanta[] llantas;
```

Esto establece una relación del tipo:

Un vehículo tiene llantas.

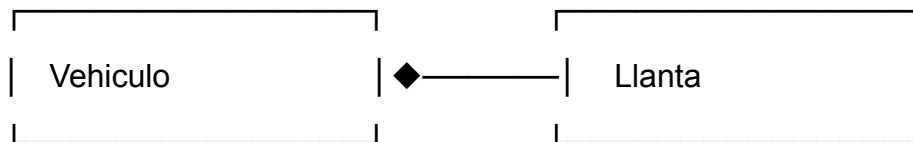
No decimos:

Un vehículo es una llanta.

Por eso no es herencia. Es una relación entre un objeto completo y una de sus partes.

Representación UML

La composición se representa con un rombo negro:



El rombo negro se coloca del lado del objeto completo:

Vehiculo ◆—— Llanta

El vehículo es el objeto completo y la llanta es una parte.

3. Dependencia: `FrmVehiculo` utiliza `Vehiculo`

Dentro del evento del botón Guardar, el formulario crea un vehículo:

Aquí `FrmVehiculo` usa temporalmente la clase `Vehiculo`.

Podemos decir:

`FrmVehiculo` depende de `Vehiculo` para registrar la información.

El formulario no necesariamente tiene un atributo permanente:

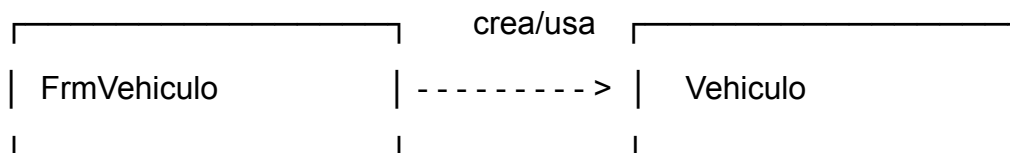
```
private Vehiculo vehiculo;
```

Simplemente crea el objeto cuando el usuario presiona el botón.

Por eso esta relación puede representarse como una **dependencia**.

Representación UML

La dependencia utiliza una línea discontinua con flecha:



La flecha apunta hacia la clase que se utiliza:

FrmVehiculo → utiliza → Vehiculo

4. **Asociación**: cuando una clase guarda una referencia a otra

La **asociación** ocurre cuando una clase conoce o mantiene una referencia a otra clase.

Por ejemplo:

```
public class FrmVehiculo extends JFrame {

    private Vehiculo vehiculoActual;

}
```

En ese caso, el formulario no solo crea una variable temporal dentro del botón. Ahora conserva el vehículo como atributo.

La relación sería:

FrmVehiculo ————— Vehiculo

La diferencia principal es:

Dependencia:

La clase utiliza temporalmente otro objeto.

Asociación:

La clase conserva una referencia como parte de sus atributos.

Comparación en código

Dependencia:

```
public void guardar() {  
    Vehiculo vehiculo = new Vehiculo(...);  
}
```

La variable solamente existe dentro del método.

Asociación:

```
public class FrmVehiculo extends JFrame {  
  
    private Vehiculo vehiculo; //VER QUE ESTA COMO VARIABLE  
    GLOBAL/ATRIBUTO  
  
    public void guardar() {  
        vehiculo = new Vehiculo(...);  
    }  
}
```

5.**Agregación:** relación “tiene”, pero con independencia

La agregación también representa una relación de:

Tiene un.

Sin embargo, es más débil que la composición.

Se representa con un rombo vacío:

Vehiculo ◇—— Llanta

Agregación: relación “tiene”, pero con independencia

La agregación también representa una relación de:

Tiene un.

Sin embargo, es más débil que la composición.

Se representa con un rombo vacío:

Vehiculo ◇—— Llanta

6.Cardinalidad o multiplicidad

Los números escritos en los extremos indican cuántos objetos participan en la relación.

Uno a uno

Vehiculo "1" —— "1" LlantaPrincipal

Estudiaten "1" ——— "1" matricula

Significa:

Cada vehículo tiene exactamente una llanta principal.

Uno a cuatro

vehiculo"1" —— "4" llantas

estudiante "1" ——— "2" telefonos

Significa:

Cada vehículo tiene exactamente cuatro llantas.

Uno a muchos

Alumno "1" — "*" tareas

El asterisco significa muchos:

Un vehículo puede tener muchas llantas.

Cero o uno

Vehiculo "1" — "0..1" LlantaRepuesto

Significa:

Un vehículo puede no tener llanta de repuesto o puede tener una.

De uno a muchos

Vehiculo "1" — "1..*" Llanta

Significa:

Debe tener como mínimo una llanta y puede tener varias.

Para este ejercicio:

Vehiculo "1" ◆ — "4" Llanta

es una representación clara.

Clases	Relación	Lectura
FrmVehiculo y JFrame	Herencia	FrmVehiculo es un JFrame
Vehiculo y Llanta	Composición	Vehiculo tiene cuatro llantas

Principal y FrmVehiculo	Dependencia	Principal crea el formulario
FrmVehiculo y Vehiculo	Dependencia	El formulario crea un vehículo
FrmVehiculo y Llanta	Dependencia	El formulario crea las llantas
FrmVehiculo y componentes Swing	Composición o asociación	El formulario contiene campos y botones

Ejercicio: Persona, mascotas y plagas

Una persona puede tener una o varias mascotas. Las mascotas pueden ser perros o gatos. Además, las mascotas pueden tener diferentes tipos de plagas, como pulgas o garrapatas.

clases:

persona

nombre

edad

dpi

mascota:

nombre

raza

edad

peso

perro:

metodos:

ladrar()

escarbar()

roncar()

gato:

-color

metodos:

maullar()

ronroneo()

plaga:

-nombre:

-niveleinfestacion:

metodos

mostrarinfo()

pulga:

-cantidadsaltos

metodos

saltar()

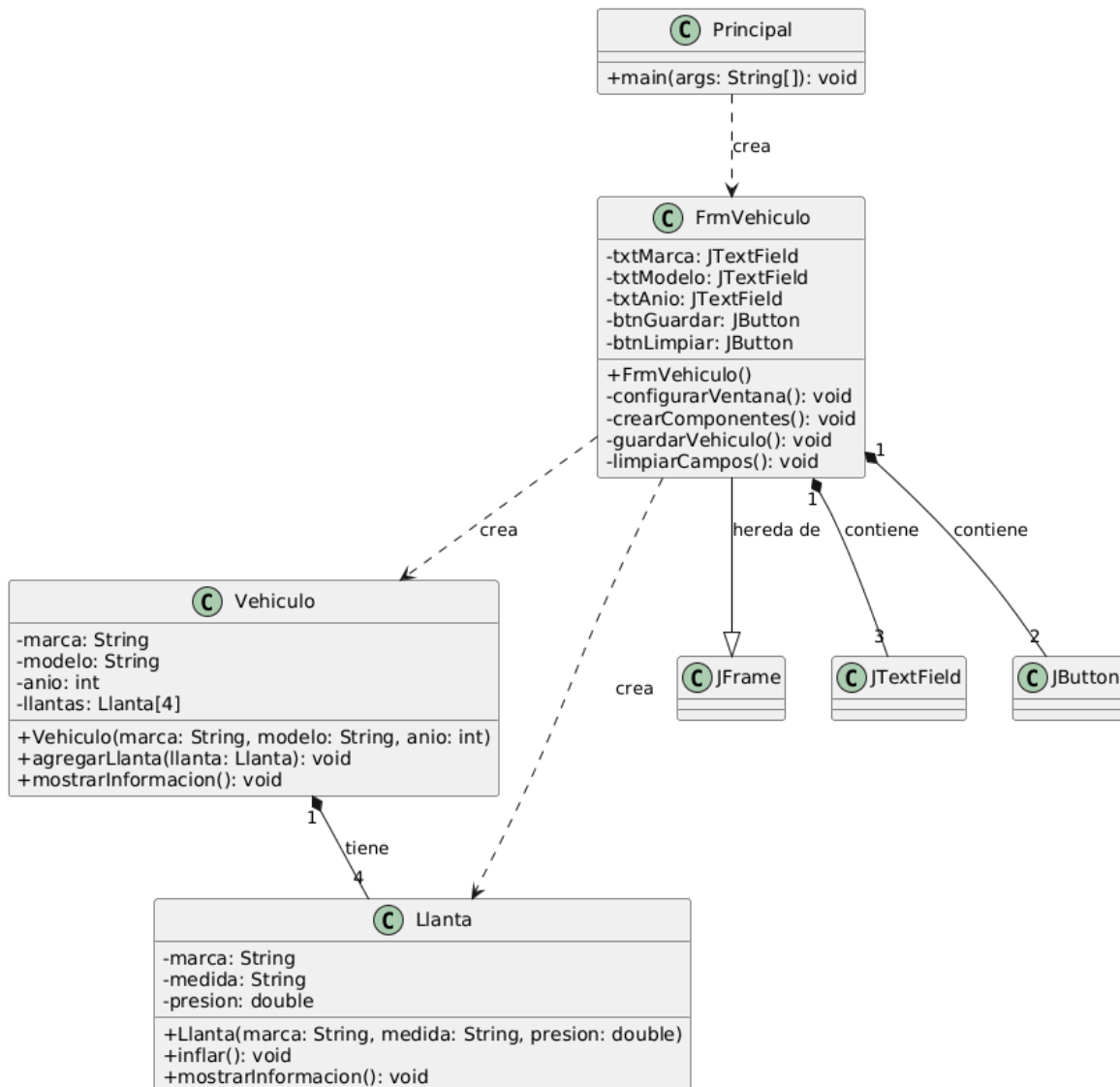
garrapata

tamanho

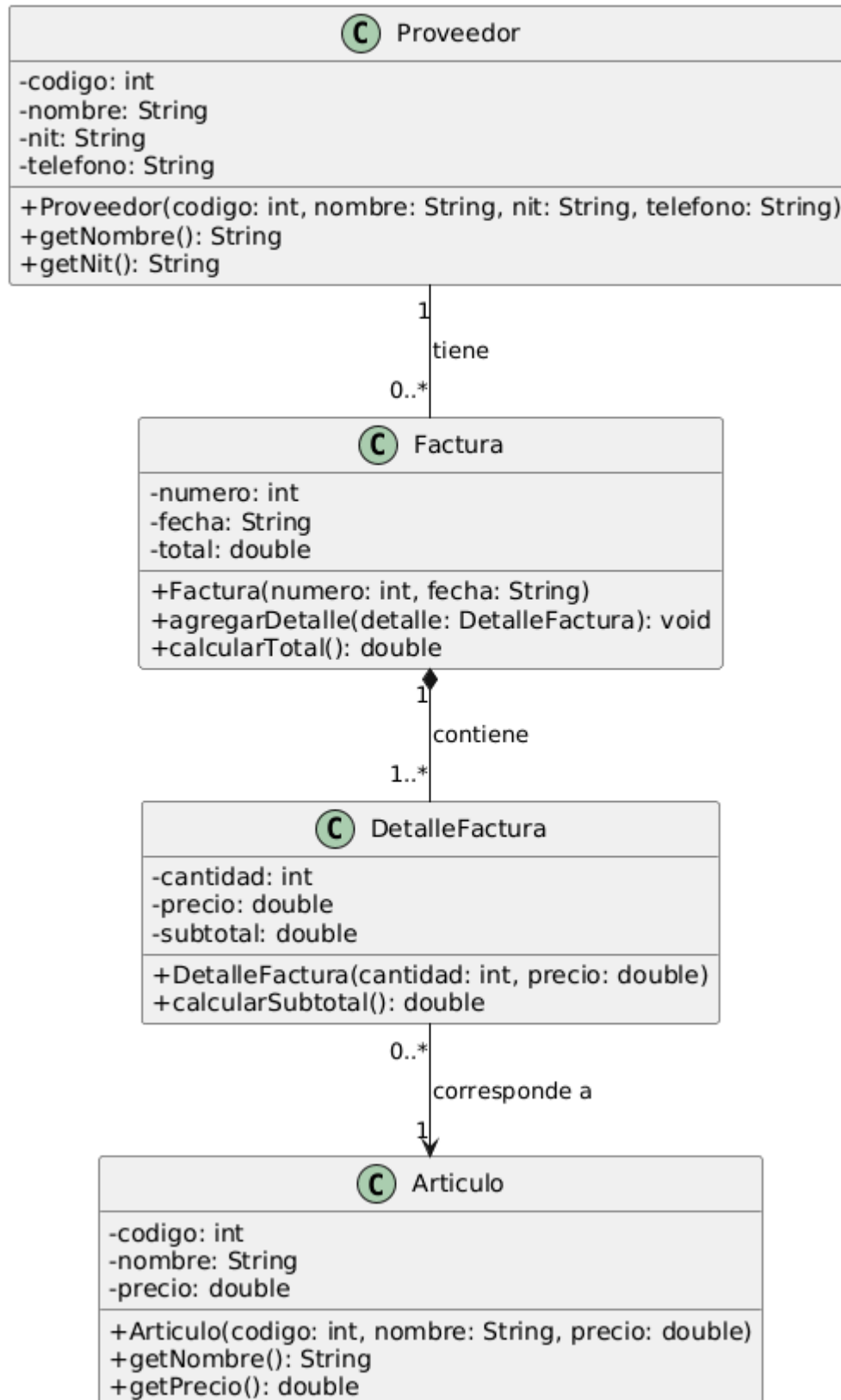
metodos

adherirse()

Sistema de registro de vehículos



Sistema de Facturación



¿Qué es un `ArrayList`?

Un `ArrayList` es una colección de Java que permite guardar varios objetos dentro de una misma variable.

En tu proyecto puede utilizarse para guardar todos los vehículos que se vayan creando desde el formulario.

A diferencia de un arreglo tradicional, un `ArrayList` puede aumentar o disminuir su tamaño

un arreglo seria asi

```
Vehiculo[] vehiculos = new Vehiculo[5];
```

pero solo podriamos almacenar 5 vehiculos que pasaria si quiero almacenar mas?

```
ArrayList<Vehiculo> vehiculos = new ArrayList<>();
```

Crear el `ArrayList` en `FrmVehiculo`

Dentro de la clase `FrmVehiculo`, crea el `ArrayList` como atributo:

```
private ArrayList<Vehiculo> listaVehiculos = new ArrayList<>();
```

y en el `addactionlistener` se debe agregar la linea asi:

```
btnGuardar.addActionListener(e -> {
```

```
    String marca = txtMarca.getText();
```

```
    String modelo = txtModelo.getText();
```

```
    int anio = Integer.parseInt(txtAnio.getText());
```

```
    double precio = Double.parseDouble(txtPrecio.getText());
```



```

        Vehiculo vehiculo = new Vehiculo(
            marca,
            modelo,
            anio,
            precio
        );
    });

//con llantas
String marcaLlanta = txtMarcaLlanta.getText();
int tamano = Integer.parseInt(txtTamano.getText());
double presion = Double.parseDouble(txtPresion.getText());

Llanta llanta = new Llanta(
    marcaLlanta,
    tamano,
    presion
);

```

como mostramos los datos que ya guardamos en el formulario?

¿Qué es un **JTable**?

Un **JTable** es un componente de Java Swing que permite mostrar información organizada en filas y columnas, como si fuera una tabla de Excel.

En tu proyecto, el **JTable** puede utilizarse para mostrar todos los vehículos que se han guardado en el **ArrayList**.

Por ejemplo:

Marca	Modelo	Año
-------	--------	-----

Toyota	Corolla	2022
--------	---------	------

Honda	Civic	2021
-------	-------	------

Mazda	3	2023
-------	---	------

Componentes principales

Para trabajar con una tabla normalmente se utilizan:

//como atributos

```
JTable tablaVehiculos;
```

```
DefaultTableModel modeloTabla;
```

```
JScrollPane scrollTabla;
```

//en el constructor:

```
tablaVehiculos = new JTable();  
modeloTabla = new DefaultTableModel();  
scrollTabla = new JScrollPane(tablaVehiculos);
```

//aqui agregamos las columnas

```
modeloTabla.addColumn("Marca");  
modeloTabla.addColumn("Modelo");  
modeloTabla.addColumn("Año");
```

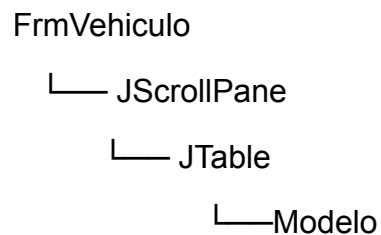
Después se asigna el modelo al **JTable**:

```
tablaVehiculos = new JTable(modeloTabla);
```

Finalmente, la tabla se coloca dentro del JScrollPane:

```
scrollTabla = new JScrollPane(tablaVehiculos);  
scrollTabla.setBounds(30, 250, 450, 180);  
add(scrollTabla);
```

por capas



Agregar elementos al jTable

nos ubicamos en el metodo addActionListener y agregamos:

```
listaVehiculos.add(vehiculo);  
modeloTabla.addRow(new Object[]{  
    vehiculo.getMarca(),  
    vehiculo.getModelo(), vehiculo.getAnio()  
});
```

Mostrar vehículos que ya existen en el **ArrayList**

Si el **ArrayList** ya contiene vehículos, podemos recorrerlo con un **for** normal y agregar cada vehículo al **JTable**.

```
private void cargarTabla() {  
  
    modeloTabla.setRowCount(0);  
  
    for (int i = 0; i < listaVehiculos.size(); i++) {  
  
        Vehiculo vehiculo = listaVehiculos.get(i);  
  
        modeloTabla.addRow(new Object[]{  
            vehiculo.getMarca(),  
            vehiculo.getModelo(),  
            vehiculo.getAnio()  
        });  
    }  
}
```

```
}
```

Explicación

Esta línea limpia las filas visibles de la tabla:

```
modeloTabla.setRowCount(0);
```

No elimina los vehículos del `ArrayList`.

El ciclo comienza en la posición `0`:

```
int i = 0;
```

Se repite mientras `i` sea menor que la cantidad de vehículos:

```
i < listaVehiculos.size();
```

En cada repetición aumenta una posición:

```
i++
```

Para obtener un vehículo según su posición usamos:

```
Vehiculo vehiculo = listaVehiculos.get(i);
```

Por ejemplo:

```
listaVehiculos.get(0);
```

obtiene el primer vehículo.

```
listaVehiculos.get(1);
```

obtiene el segundo vehículo.

Después agregamos sus datos como una nueva fila:

```
modeloTabla.addRow(new Object[]{  
    vehiculo.getMarca(),  
    vehiculo.getModelo(),  
    vehiculo.getAnio()
```

});